

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

TOPIC: C++ Programming Language

Prepared By: Kirti Dethé

1. Introduction

C++ is a general-purpose programming language used to develop different types of software and applications.

C++ supports both **procedural programming** and **object-oriented programming (OOP)**. It is known for its speed, efficiency, and ability to work closely with computer hardware.

C++ is also used in embedded systems and Arduino-related programming. Your uploaded material notes that Arduino programs use a **simple C/C++-based language**

2. History of C++

- C++ was developed by **Bjarne Stroustrup**.
- It was developed at **Bell Labs**.
- C++ was originally known as “**C with Classes**.”
- It was designed as an extension of the C programming language.
- The language later became known as **C++**

3. Features of C++

1. Object-Oriented

C++ supports OOP concepts such as:

- Class
- Object
- Inheritance
- Polymorphism
- Encapsulation
- Abstraction

2. Fast

C++ is a compiled language and can provide high performance.

3. Portable

C++ programs can generally be adapted to different operating systems and platforms.

4. Structured

Programs can be divided into functions and logical sections.

5. Rich Library

C++ provides many standard libraries for common programming tasks.

6. Low-Level Access

C++ provides features such as pointers that allow programmers to work closely with memory and hardware.

4. Applications of C++

C++ is used in:

- Operating systems
- Game development
- Embedded systems
- Robotics
- Desktop applications
- Browsers
- Database software
- High-performance applications
- Competitive programming
- Computer graphics

5. Basic Structure of a C++ Program

C++ Code

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      cout << "Hello World";
7      return 0;
8  }
```

Output

```
C:\> g++ hello.cpp -o hello
C:\> hello
Hello World
```

Explanation:

1. `#include <iostream>` – Provides input and output functionality (like `cout` and `cin`).
2. `using namespace std;` – Allows us to use standard library names without writing `std::`.
3. `int main()` – The main function where the program starts.
4. `cout << "Hello World";` – Displays the text “Hello World” on the screen.
5. `return 0;` – Indicates successful completion of the program.

6. Variables

A variable is a named memory location used to store data.

Example:

```
int age = 20;
float marks = 85.5;
char grade = 'A';
```

Here:

- `age` stores an integer.
- `marks` stores a decimal value.
- `grade` stores a character.

7. Data Types

Common C++ data types include:

C++ Data Types

A data type tells the compiler what kind of value a variable can store. C++ provides several built-in data types to handle different kinds of data.

Data Type	Size (approx.)	Example	Description
int	4 bytes	10	Used to store whole numbers (integers).
float	4 bytes	10.5	Used to store decimal numbers (single precision).
double	8 bytes	25.6789	Used to store decimal numbers (double precision).
char	1 byte	'A'	Used to store a single character.
bool	1 byte	true / false	Used to store Boolean values (true or false).
string	Variable	"Kirti"	Used to store a sequence of characters (text).

Example Code

```
#include <iostream>
using namespace std;

int main() {
    int age = 20;
    float marks = 85.5;
    char grade = 'A';
    bool isPass = true;
    string name = "Kirti";
    cout << "Age: " << age << endl;
    cout << "Marks: " << marks << endl;
    cout << "Grade: " << grade << endl;
    cout << "Is Pass: " << isPass << endl;
    cout << "Name: " << name << endl;
    return 0;
}
```

Output

```
Age: 20
Marks: 85.5
Grade: A
Is Pass: 1
Name: Kirti
```



Important Points

- int is used for whole numbers.
- float and double are used for decimal numbers.
- char is used for single characters.
- bool stores true or false.
- string is used for text (requires #include <string> in some compilers).

8. Input and Output

Output – cout

```
cout << "Welcome to C++";
```

Input – cin

```
int age;  
cin >> age;
```

Complete example:

```
#include <iostream>  
using namespace std;  
  
int main()  
{  
    int age;  
  
    cout << "Enter your age: ";  
    cin >> age;  
  
    cout << "Your age is: " << age;  
  
    return 0;  
}
```

9. Operators

C++ provides different types of operators.

Arithmetic Operators

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

Example:

```
int a = 10, b = 3;  
  
cout << a + b;  
cout << a - b;  
cout << a * b;  
cout << a / b;  
cout << a % b;
```

10. Conditional Statements

Conditional statements are used to make decisions.

if-else

```
#include <iostream>
using namespace std;

int main()
{
    int marks;

    cout << "Enter marks: ";
    cin >> marks;

    if (marks >= 40)
        cout << "Pass";
    else
        cout << "Fail";

    return 0;
}
```

Working

Enter Marks

```
      ↓
Marks >= 40?
  ↓       ↓
Yes       No
  ↓       ↓
Pass      Fail
```

11. Loops

Loops are used to execute a block of code repeatedly.

for loop

```
#include <iostream>
using namespace std;

int main()
{
    for(int i = 1; i <= 5; i++)
    {
        cout << i << endl;
    }

    return 0;
}
```

Output:

```
1
2
3
4
5
```

Other loops include:

- while
- do-while
- for

12. Functions

A function is a block of code designed to perform a particular task.

Example:

```
#include <iostream>
using namespace std;

void display()
{
    cout << "Hello C++";
}

int main()
{
    display();

    return 0;
}
```

Advantages

- Code reusability
 - Easy debugging
 - Better program organization
 - Reduces repetition
-

13. Array

An array stores multiple values of the same data type.

Example:

```
#include <iostream>
using namespace std;

int main()
{
    int marks[5] = {70, 80, 65, 90, 75};

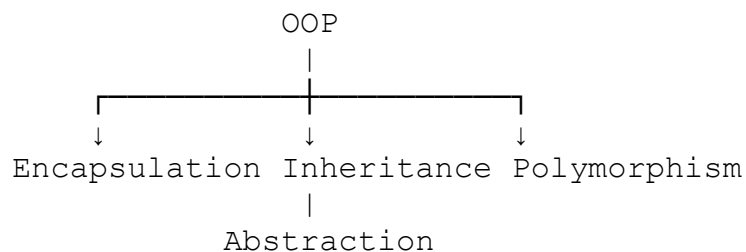
    for(int i = 0; i < 5; i++)
    {
        cout << marks[i] << endl;
    }

    return 0;
}
```

14. Object-Oriented Programming

One of the most important parts of C++ is **Object-Oriented Programming (OOP)**.

The major OOP concepts are:



15. Class

A **class** is a blueprint or template for creating objects.

Example:

```
class Student
{
public:
    string name;
    int rollNo;
};
```

16. Object

An **object** is an instance of a class.

Example:

```
Student s1;
```

```
s1.name = "Kirti";
```

```
s1.rollNo = 101;
```

Complete example:

```
#include <iostream>
using namespace std;
```

```
class Student
{
public:
    string name;
    int rollNo;

    void display()
    {
        cout << "Name: " << name << endl;
        cout << "Roll No: " << rollNo << endl;
    }
};
```

```
int main()
{
    Student s1;

    s1.name = "Kirti";
    s1.rollNo = 101;

    s1.display();

    return 0;
}
```

17. Encapsulation

Encapsulation means combining data and functions into a single unit, usually a class, while controlling access to the data.

Example:

```
class Student
{
private:
    int marks;

public:
    void setMarks(int m)
    {
        marks = m;
    }

    int getMarks()
    {
        return marks;
    }
};
```

18. Inheritance

Inheritance allows one class to acquire properties and functions of another class.

Example:

```
class Animal
{
public:
    void eat()
    {
        cout << "Eating";
    }
};

class Dog : public Animal
{
public:
    void bark()
    {
        cout << "Barking";
    }
};
```

Here, Dog inherits from Animal.

19. Polymorphism

Polymorphism means **one interface/name can have different forms or behaviors**.

A common example is function overloading:

```
class Calculator
{
public:
    int add(int a, int b)
    {
        return a + b;
    }

    int add(int a, int b, int c)
    {
        return a + b + c;
    }
};
```

The function name add() is the same, but it works with different parameters.

20. Abstraction

Abstraction means showing the important information while hiding unnecessary implementation details.

For example, when using a calculator, we use buttons to perform operations without needing to know the internal circuitry or calculations.

21. Pointers

A pointer is a variable that stores the **address of another variable**.

Example:

```
int a = 10;

int *p = &a;

cout << *p;
```

Output:

10

22. Advantages of C++

- High performance
- Supports OOP
- Supports procedural programming
- Provides memory control
- Suitable for large applications
- Used in game development
- Useful for embedded systems
- Provides a large standard library

23. Limitations of C++

- More complex than some beginner-oriented languages
- Manual memory management can cause errors
- Pointers can be difficult for beginners
- Large programs can become complicated if poorly designed

24. Simple C++ Mini Project

Student Marks Calculator

```
#include <iostream>
using namespace std;

int main()
{
    string name;
    float m1, m2, m3, total, percentage;

    cout << "Enter student name: ";
    cin >> name;

    cout << "Enter marks of 3 subjects: ";
    cin >> m1 >> m2 >> m3;

    total = m1 + m2 + m3;
    percentage = total / 3;

    cout << "\nStudent Name: " << name;
    cout << "\nTotal Marks: " << total;
    cout << "\nPercentage: " << percentage << "%";

    if (percentage >= 40)
```

```
        cout << "\nResult: Pass";  
    else  
        cout << "\nResult: Fail";  
  
    return 0;  
}
```